

# Proposta de Arquitetura Técnica de Referência

**Preparado para:** Jabil **Elaborado por:** Victor Lucas — Engenheiro Fullstack Sênior **Data:** 14/07/2026 (revisão 3 — ajustada em resposta ao parecer de aderência à Guideline de TI da EMS IT)

## Sumário executivo

Este documento descreve o padrão de arquitetura e a stack tecnológica adotados no projeto de referência **Bil**, hoje direcionado ao domínio da Jabil: um **dashboard de OEE (Overall Equipment Effectiveness) para linha de produção SMT**, com autenticação de usuários e acompanhamento de paradas (downtime). Diferente da primeira versão deste documento — que era uma proposta especulativa, escrita antes de existir código —, esta revisão descreve **o que já está implementado no repositório**, o que é apenas mock/fachada no frontend, e o que ainda falta construir no backend.

**O que mudou nesta revisão:** a Jabil retornou um parecer formal de aderência à **Guideline de TI da EMS IT**, item a item (arquitetura, qualidade, segurança, observabilidade, auditoria, i18n, CI/CD). A seção 6 traz essa tabela na íntegra com o status recebido, e as seções 2 a 5 foram ajustadas para incorporar cada adequação exigida como item de trabalho explícito — não como recomendação genérica de "boas práticas", mas como requisito de aceite do cliente.

Resumo em uma frase: **separamos claramente "regra de negócio" de "detalhe técnico"** (banco, cache, framework), o que nos dá testes rápidos e confiáveis, facilidade para trocar peças da infraestrutura sem reescrever o sistema, e um código mais previsível para qualquer engenheiro que entrar no time depois.

**Importante:** as versões listadas abaixo são as que estão de fato instaladas no `package.json` do monorepo hoje — não as versões "mais recentes disponíveis". Onde há um upgrade recomendado e ainda não feito, isso está marcado explicitamente na seção 5.

## 1. Visão geral da arquitetura

Usamos **Clean Architecture + DDD tático** no backend e uma SPA React orientada a **estado de servidor** no frontend. A regra que guia todas as decisões abaixo:

**As dependências apontam sempre para dentro.** O núcleo de regras de negócio não sabe que Postgres, Redis ou NestJS existem — ele só conhece interfaces (contratos).

```
infra → domain → core
(detalhe)  (regra)  (genérico)
```

No backend isso já existe fisicamente no repositório: `backend/src/core` , `backend/src/domain` , `backend/src/infra` .

Por que isso importa para o negócio, não só para o time técnico:

- **Testes não dependem de banco/infra** → a suíte de testes de use case usa repositórios fake em memória (ver `test/repositories` ), então roda em segundos.
- **Trocar Postgres por outro banco, ou Redis por outro cache, não obriga a reescrever regra de negócio** — só a peça de infraestrutura.
- **Regra de negócio fica em um único lugar** (a entidade/use case), não espalhada em controllers.

## 2. Backend

### 2.1 Stack (versões reais em `backend/package.json` )

| Área           | Tecnologia                                                                                                                                                     | Versão instalada                                                                                                                                                                              | Observação                                                                                                                  |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Framework      | NestJS                                                                                                                                                         | 10.x                                                                                                                                                                                          | Módulos, DI e decorators nativos resolvem validação, guards e interceptors sem reinventar em cima do Express puro           |
| Linguagem      | TypeScript                                                                                                                                                     | 5.1                                                                                                                                                                                           | <code>target: es2022</code> , <code>strict: true</code>                                                                     |
| ORM / Banco    | Prisma                                                                                                                                                         | 5.2 + PostgreSQL 17 ( <code>docker-compose.yml</code> )                                                                                                                                       | Schema hoje tem só a tabela <code>users</code> — o domínio de OEE/downtime ainda não tem persistência própria (ver seção 5) |
| Cache          | Redis 7 (alpine) + <b>ioredis 5.3</b>                                                                                                                          | <code>RedisCacheRepository</code> implementa uma interface <code>CacheRepository</code> genérica em <code>core/cache</code> — cache-aside, TTL de 15 min                                      |                                                                                                                             |
| Autenticação   | JWT <b>RS256</b> via <code>@nestjs/jwt</code> + <code>passport-jwt</code>                                                                                      | Chave assimétrica ( <code>JWT_PRIVATE_KEY</code> / <code>JWT_PUBLIC_KEY</code> em base64), guard global ( <code>JwtAuthGuard</code> ) com rotas públicas via decorator <code>@Public()</code> |                                                                                                                             |
| Validação      | <b>Zod v3</b> ( <code>^3.22</code> ) + <code>zod-validation-error</code>                                                                                       | Valida <code>env</code> ( <code>envSchema</code> ) e payload do JWT                                                                                                                           |                                                                                                                             |
| Upload/Storage | Storage local em disco ( <code>DiskStorage</code> , <code>UPLOADS_DIRECTORY</code> ) atrás de uma interface <code>Uploader</code> em <code>core/storage</code> | Não há integração com S3/R2 hoje — a interface já existe para isso ser trocado sem tocar em regra de negócio                                                                                  |                                                                                                                             |

| Área            | Tecnologia                                                                                 | Versão instalada                                                          | Observação |
|-----------------|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|------------|
| Testes          | <b>Vitest 0.34</b> (unit) / config separada para e2e ( <code>vitest.config.e2e.ts</code> ) |                                                                           |            |
| Package manager | <b>pnpm</b> ( <code>packageManager: pnpm@11.10.0</code> ) via <b>Turborepo 2.5</b>         | Monorepo com <code>backend</code> e <code>frontend</code> como workspaces |            |

## 2.2 Either — erro de negócio como valor, não como exceção

Use cases não usam `throw` para erro de regra de negócio. Retornam um tipo `Either<Erro, Sucesso>` ( `backend/src/core/either.ts` ):

```
export class Left<L, R> {
  constructor(readonly value: L) {}
  isLeft(): this is Left<L, R> { return true }
  isRight(): this is Right<L, R> { return false }
}

export class Right<L, R> {
  constructor(readonly value: R) {}
  isLeft(): this is Left<L, R> { return false }
  isRight(): this is Right<L, R> { return true }
}

export type Either<L, R> = Left<L, R> | Right<L, R>
```

**Por quê:** o próprio tipo de retorno do use case já documenta todo erro possível daquela operação. O TypeScript obriga quem chama a checar `isLeft()` antes de usar o valor.

**Adequação exigida pela EMS IT (item "Tratamento de exceções" — Parcial):** o Either cobre erro de regra de negócio esperado, mas a guideline pede tratamento padronizado também para exceção *inesperada* (falha de infra, bug, etc.). Isso ainda não existe no projeto e entra como item de trabalho: um `GlobalExceptionHandler` ( `@Catch()` do NestJS) capturando qualquer exceção não tratada, padronizando o corpo de resposta HTTP (código, mensagem amigável, `traceId` ), logando automaticamente o stack trace e nunca vazando detalhe interno para o cliente. O padrão Either continua sendo a forma correta de modelar erro de negócio dentro do use case — o filtro global é a rede de segurança para o que escapar dele.

## 2.3 Use cases e repositórios como interface

Exemplo real do domínio hoje implementado ( `domain/user/application/use-cases/authenticate-user.ts` ):

```
@Injectable()
export class AuthenticateUserUseCase {
  constructor(
    private usersRepository: UsersRepository, // interface, não implementação
    private hashComparer: HashComparer,
```

```

    private encrypter: Encrypter,
  ) {}

  async execute({ email, password }: AuthenticateUserUseCaseRequest) {
    const user = await this.usersRepository.findByEmail(email)
    if (!user) return left(new WrongCredentialsError())

    const isPasswordValid = await this.hashComparer.compare(password, user.password)
    if (!isPasswordValid) return left(new WrongCredentialsError())

    const accessToken = await this.encrypter.encrypt({ sub: user.id.toString() })
    return right({ accessToken })
  }
}

```

`UsersRepository` é uma classe abstrata definida no `domain`. Quem implementa de verdade — com Prisma, ou um fake em memória para teste — mora em `infra / test/repositories`. O use case nunca importa Prisma.

**Ganho concreto:** `authenticate-user.spec.ts` e `register-user.spec.ts` rodam com repositório, hasher e encrypter fake — sem subir Postgres nem Redis.

## 2.4 Eventos de domínio

A infraestrutura de eventos de domínio ( `core/events/domain-events.ts` , `AggregateRoot.addDomainEvent` ) já existe no `core` — é a mesma base usada em projetos anteriores de referência — mas **hoje nenhum use case do domínio user dispara evento nenhum**, porque não há mais um segundo módulo (o antigo domínio de fórum/notificações, que consumia esses eventos, foi removido do projeto). Fica como peça pronta para quando o domínio de OEE precisar, por exemplo, notificar alguém quando uma parada de linha ultrapassar um limite de tempo.

## 2.5 Segurança — adequações exigidas pela EMS IT

A autenticação hoje (JWT RS256 com chave assimétrica, guard global) atende ao básico, mas o parecer da EMS IT classifica o item **Segurança como "Parcial"** e pede complementos que ainda não existem no projeto:

- **RBAC (Role-Based Access Control):** hoje qualquer usuário autenticado acessa qualquer rota protegida. Falta modelar papéis (ex.: operador de linha, supervisor, admin) e um guard/decorator de autorização por papel.
- **OAuth2/OpenID Connect:** relevante se a Jabil exigir SSO corporativo (Azure AD/Entra ID, por exemplo) em vez de login local — a decidir com o time de infra da Jabil se o login atual (email/senha) é suficiente ou se precisa federar com o IdP corporativo.
- **Gerenciamento de segredos:** `JWT_PRIVATE_KEY` / `JWT_PUBLIC_KEY` e credenciais de banco hoje vivem em variáveis de ambiente ( `.env` ). Falta migrar para um cofre gerenciado (Azure Key Vault ou AWS Secrets Manager) antes de produção, com rotação de chaves.
- **Rate limiting:** não implementado. Adicionar `@nestjs/throttler` (ou equivalente) nas rotas de autenticação, no mínimo, para mitigar força bruta.
- **Proteção OWASP Top 10:** validação de entrada já existe via Zod, mas falta checklist explícito (headers de segurança com `helmet`, CORS restritivo por ambiente, proteção contra IDOR nas rotas de OEE/downtime quando existirem).

- **Auditoria de autenticação/autorização:** ver item "Auditoria" na seção 5 — tentativas de login, falhas e mudanças de permissão precisam ficar em log de auditoria, não só em log de aplicação.

Nenhum desses itens é bloqueante para o estado atual (só há domínio `user`), mas todos precisam estar resolvidos antes de o domínio de OEE ir para produção com dado real de planta.

### 3. Frontend

#### 3.1 Stack (versões reais em `frontend/package.json`)

| Área                        | Tecnologia                                                                                                  | Versão instalada                                                                                                                           | Observação                             |
|-----------------------------|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| Framework                   | React                                                                                                       | 18.2                                                                                                                                       |                                        |
| Build tool                  | Vite                                                                                                        | 5.0                                                                                                                                        |                                        |
| Linguagem                   | TypeScript                                                                                                  | 5.2                                                                                                                                        | <code>moduleResolution: bundler</code> |
| Roteamento                  | React Router DOM                                                                                            | 6.21                                                                                                                                       |                                        |
| Estado de servidor          | TanStack Query                                                                                              | 5.17                                                                                                                                       |                                        |
| Formulários                 | <code>react-hook-form 7.49</code> + <code>@hookform/resolvers</code> + <code>Zod v3</code>                  |                                                                                                                                            |                                        |
| Design system               | <code>shadcn/ui</code> sobre <b>Radix UI</b> ( <code>@radix-ui/react-*</code> diretos, não um pacote único) | Componentes copiados para <code>src/components/ui</code> , não importados de um pacote fechado                                             |                                        |
| Estilo                      | Tailwind CSS 3.3                                                                                            | <code>tailwind.config.js</code> tradicional (não é a config em CSS do Tailwind v4)                                                         |                                        |
| Ícones                      | <code>lucide-react</code>                                                                                   |                                                                                                                                            |                                        |
| Testes unitários/componente | Vitest 1.2 + Testing Library                                                                                |                                                                                                                                            |                                        |
| Testes e2e                  | Playwright 1.41                                                                                             |                                                                                                                                            |                                        |
| Mock de API                 | MirageJS 0.1.48 ( <code>src/api/mirage</code> )                                                             | Intercepta as chamadas do <code>axios</code> em desenvolvimento — hoje é a única fonte de dados para as telas de OEE/downtime, ver seção 5 |                                        |
| Lint                        | ESLint 8 (config <code>.eslintrc</code> , não flat config)                                                  |                                                                                                                                            |                                        |

#### 3.2 Design system: `shadcn/ui` + Radix

Seguimos o padrão shadcn (copiar o componente para o repo, não importar de um pacote fechado): o time mantém controle total de estilo e comportamento, sem depender de release de terceiro para customizar um componente. Hoje o motor por baixo é **Radix UI** — a migração para Base UI (motor mantido pelo time do MUI, que o shadcn passou a suportar como alternativa) é uma opção de upgrade futuro, não algo já feito neste projeto.

### 3.3 Estado de servidor e o domínio de OEE

Todo dado que vem da API passa por `useQuery / useMutation` do TanStack Query — não guardamos resposta de API em `useState` global nem em Context solto. O cliente HTTP único é o `bilApi` ( `frontend/src/lib/bil-api.ts` ), uma instância de `axios` com `withCredentials: true` e um interceptor opcional de delay artificial ( `VITE_ENABLE_API_DELAY` ) para simular latência de rede em desenvolvimento.

As telas de negócio hoje implementadas:

- **Dashboard** ( `pages/app/dashboard` ): KPI cards de OEE/disponibilidade/performance/qualidade/produção, gráfico de tendência de OEE, composição do OEE, OEE por equipamento e Pareto de paradas — todos filtráveis por linha e período ( `DashboardFilters` ).
- **Downtime** ( `pages/app/downtime` ): histórico de paradas, filtro por tabela, e tela de detalhe/resolução de um evento de parada.
- **Auth** ( `pages/auth` ): login e cadastro.

Cada tela consome uma função em `src/api/*.ts` (ex.: `get-ooo-overview.ts` , `get-downtime-pareto.ts` ) tipada de ponta a ponta com a resposta esperada da API.

### 3.4 Formulários

```
const schema = z.object({
  email: z.string().email(),
})

const { register, handleSubmit, formState: { isSubmitting } } = useForm({
  resolver: zodResolver(schema),
})
```

Validação client-side e submissão desabilitada durante o envio ( `isSubmitting` ) evitam duplo clique e dado inválido chegando na API.

## 4. Ferramentas transversais

| Item                       | Escolha                            | Situação                                                                              |
|----------------------------|------------------------------------|---------------------------------------------------------------------------------------|
| Package manager / monorepo | <b>pnpm workspaces + Turborepo</b> | Já configurado ( <code>pnpm dev/build/lint/test</code> na raiz rodam os dois pacotes) |
| Formatação                 | Prettier                           | Mantido                                                                               |

| Item                       | Escolha                                                                                                                                    | Situação                                                                                                                                                                                                                                                                          |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Qualidade estática         | Vitest + ESLint + Prettier hoje;<br><b>SonarQube/SonarCloud exigido pela guideline EMS IT</b>                                              | <b>Não atende ainda</b> — falta integrar ao pipeline com Quality Gate (cobertura mínima, duplicação, complexidade ciclomática, SAST) bloqueando merge para a branch principal                                                                                                     |
| CI/CD                      | GitHub Actions previsto                                                                                                                    | <b>Não configurado</b> — não existe <code>.github/workflows</code> no repositório hoje. A EMS IT usa <b>Azure DevOps</b> : pipeline alvo é Build → Testes → SonarQube → Dependency/Security Scan (SAST) → artefato versionado → aprovação de release → deploy DEV/QA/UAT/Produção |
| Logging corporativo        | <code>ConsoleLogger</code> padrão do Nest                                                                                                  | <b>Parcial</b> — falta logging estruturado (Winston ou Pino), níveis (Info/Warning/Error/Debug), Correlation ID/Request ID por requisição, e integração com a solução de monitoramento corporativa (Azure Monitor/CloudWatch/ELK)                                                 |
| Internacionalização (i18n) | Não prevista hoje (textos hardcoded em PT-BR)                                                                                              | <b>Não atende</b> — a guideline exige i18n; entra como item de trabalho: <code>react-i18next</code> no frontend, <code>Accept-Language</code> no backend, e migração de toda mensagem/validação para arquivo de recurso                                                           |
| Observabilidade            | —                                                                                                                                          | <b>Não atende</b> — a definir com o time de infra da Jabil: OpenTelemetry, health checks, métricas (Prometheus), dashboards (Grafana/Azure Monitor) e distributed tracing                                                                                                         |
| Auditoria                  | —                                                                                                                                          | <b>Não atende</b> — falta um Audit Trail (usuário, timestamp, operação, dado antes/depois, origem da requisição) para as operações críticas do domínio de OEE/downtime                                                                                                            |
| Infra local                | Docker Compose ( <code>docker-compose.yml</code> na raiz) sobe só Postgres 17 e Redis 7; backend e frontend rodam localmente via Turborepo | Funcional hoje                                                                                                                                                                                                                                                                    |

## 5. O que fica em aberto / próximos passos

Esta seção incorpora as adequações apontadas no parecer de aderência à Guideline EMS IT (seção 6), priorizadas junto com os gaps já conhecidos de domínio.

- Backend do domínio de OEE/downtime não existe ainda.** O schema Prisma só tem `users`. Todas as métricas de OEE, o Pareto de paradas e o histórico de downtime que aparecem no frontend hoje vêm do **mock MirageJS** ( `frontend/src/api/mirage` ), não de um banco real. Este é o próximo módulo de domínio a modelar (entidades de linha, equipamento, evento de parada, e os cálculos de  $OEE = disponibilidade \times performance \times qualidade$ ).
- SonarQube/SonarCloud no pipeline (Não atende).** Integrar ao Azure DevOps com Quality Gate: cobertura mínima de teste, complexidade ciclomática, duplicação de código e SAST bloqueando merge para a branch principal.

- 3. **Logging estruturado (Parcial).** Trocar o `ConsoleLogger` por Winston ou Pino, com níveis (Info/Warning/Error/Debug), Correlation ID/Request ID propagado entre requisições, e integração com a solução corporativa de monitoramento.
- 4. **Tratamento global de exceções (Parcial).** Implementar `GlobalExceptionHandler` do NestJS complementando o padrão Either (ver seção 2.2), padronizando resposta HTTP e logando automaticamente exceção não tratada.
- 5. **Internacionalização — i18n (Não atende).** `react-i18next` no frontend e `Accept-Language` no backend, externalizando toda mensagem/validação para arquivo de recurso.
- 6. **Observabilidade (Não atende).** OpenTelemetry, health checks, métricas (Prometheus) e dashboards (Grafana/Azure Monitor), com distributed tracing — decidir ferramenta final com o time de infra da Jabil.
- 7. **Auditoria (Não atende).** Audit Trail para operações críticas do domínio de OEE/downtime: usuário, timestamp, operação, dado antes/depois, origem da requisição.
- 8. **Segurança complementar (Parcial).** RBAC, avaliação de OAuth2/OIDC com IdP corporativo, cofre de segredos (Key Vault/Secrets Manager), rate limiting e checklist OWASP Top 10 — detalhado na seção 2.5.
- 9. **CI/CD completo (Parcial).** Pipeline no Azure DevOps (não GitHub Actions, para alinhar com a ferramenta já usada pela EMS IT): Build, Testes, SonarQube, Dependency/Security Scan, geração de artefato versionado, aprovação de release e deploy automatizado entre DEV/QA/UAT/Produção.
- 10. **Documentação equivalente à XML (N/A, mas recomendado).** TypeDoc/JSDoc + OpenAPI (Swagger) como equivalente ao padrão de documentação XML do .NET, já que o projeto é TypeScript/NestJS.
- 11. **Storage é local em disco.** A interface `Uploader` já isola isso, mas se houver necessidade de anexar evidência/foto a um evento de parada em produção, trocar para um storage compatível com S3 (ex.: Cloudflare R2) é só implementar a interface — sem tocar em use case.
- 12. **Upgrade de dependências.** As versões atuais (Nest 10, React 18, Tailwind 3, Zod 3, Radix direto) são estáveis mas não são as mais recentes do ecossistema em 2026. Não é bloqueante, mas vale planejar uma janela de upgrade (Tailwind v4, Zod v4, Base UI) antes do projeto crescer muito, para não acumular dívida de migração.
- 13. **Eventos de domínio sem consumidor.** A infraestrutura de `DomainEvents` está pronta e sem uso — decidir se o alerta de "parada de linha longa" deve nascer como evento de domínio (desacoplado) ou como lógica direta no use case, e se o Audit Trail (item 7) deve ser um dos consumidores desses eventos.
- 14. **Confirmar com a Jabil** se há restrição de stack já homologada (ex.: banco de dados obrigatório, provedor de nuvem, IdP corporativo para SSO) que precise ajustar as escolhas acima.

## 6. Aderência à Guideline de TI da EMS IT — parecer recebido

Tabela recebida da Jabil, item a item, comparando a Guideline de TI da EMS IT com a arquitetura proposta neste documento. Mantida na íntegra para rastreabilidade; as adequações aqui já estão refletidas nas seções 2 a 5 acima.

| Item da Guideline EMS IT       | Arquitetura Proposta     | Aderência | Adequações Necessárias |
|--------------------------------|--------------------------|-----------|------------------------|
| Arquitetura em camadas         | Clean Architecture + DDD | Atende    | —                      |
| Separação de responsabilidades | Core / Domain / Infra    | Atende    | —                      |
| Dependency Injection           | NestJS DI                | Atende    | —                      |

| Item da Guideline EMS IT   | Arquitetura Proposta                              | Aderência  | Adequações Necessárias                                                                                                                                                                                   |
|----------------------------|---------------------------------------------------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Padrões de Projeto (GoF)   | Repository, Factory, Strategy, Adapter implícitos | Atende     | —                                                                                                                                                                                                        |
| Qualidade de código        | Vitest, ESLint, Prettier                          | Parcial    | Complementar com SonarQube (SAST), métricas de cobertura, complexidade ciclomática, duplicação de código e políticas de aprovação obrigatórias no CI/CD.                                                 |
| SonarQube obrigatório      | Não citado                                        | Não atende | Implementar SonarQube/SonarCloud integrado ao Azure DevOps Pipeline, com Quality Gates e validação obrigatória antes do merge para a branch principal.                                                   |
| Logging corporativo        | Não definido                                      | Parcial    | Logging estruturado (Winston/Pino), níveis de log, Correlation ID, Request ID, centralização e integração com Azure Monitor/CloudWatch/ELK.                                                              |
| Tratamento de exceções     | Either Pattern                                    | Parcial    | Manter Either para regra de negócio; adicionar Global Exception Filter do NestJS para exceção inesperada, resposta HTTP padronizada e log automático.                                                    |
| Internacionalização (i18n) | Não prevista                                      | Não atende | react-i18next no frontend, <b>Accept - Language</b> no backend, mensagens externalizadas em arquivo de recurso.                                                                                          |
| Documentação XML           | Não aplicável (Node)                              | N/A        | Usar TypeDoc, JSDoc e OpenAPI (Swagger) como equivalente.                                                                                                                                                |
| Convenções de nomenclatura | PascalCase/CamelCase                              | Atende     | —                                                                                                                                                                                                        |
| Banco ACID                 | PostgreSQL                                        | Atende     | —                                                                                                                                                                                                        |
| CI/CD                      | Planejado                                         | Parcial    | Pipeline completo no Azure DevOps: Build, Testes, SonarQube, Dependency/Security Scan, artefatos versionados, aprovação de release, deploy DEV/QA/UAT/Produção.                                          |
| Code Review                | Previsto                                          | Atende     | —                                                                                                                                                                                                        |
| Testes automatizados       | Unit + E2E                                        | Atende     | —                                                                                                                                                                                                        |
| Observabilidade            | Não definida                                      | Não atende | OpenTelemetry, Health Checks, métricas (Prometheus), dashboards (Grafana/Azure Monitor), Distributed Tracing.                                                                                            |
| Auditoria                  | Não definida                                      | Não atende | Audit Trail: usuário, data/hora, operação, dado antes/depois, origem da requisição, para operações críticas.                                                                                             |
| Segurança                  | JWT RS256                                         | Parcial    | OAuth2/OIDC quando aplicável, RBAC, cofre de segredos (Key Vault/Secrets Manager), Rate Limiting, checklist OWASP Top 10, rotação de chaves, criptografia de dados sensíveis, auditoria de autenticação. |

Por que essa arquitetura, em uma frase para cada camada

- **Backend em Clean Architecture/DDD**: testamos regra de negócio sem infraestrutura, e trocamos peça de infraestrutura (storage, cache, banco) sem reescrever regra de negócio — hoje só o domínio `user` está implementado, mas o mesmo molde já vale para o domínio de OEE quando for construído.
- **Frontend em React + TanStack Query**: a tela sempre reflete o estado real do servidor (ou do mock, enquanto o backend de OEE não existe), com cache e updates otimistas resolvidos por uma lib madura em vez de código artesanal.
- **Design system shadcn (Radix) + Tailwind**: componentes acessíveis por padrão, 100% customizáveis, sem depender de release de terceiro para mudar comportamento.
- **pnpm + Turborepo + TypeScript + Vitest no full stack**: um único conjunto de ferramentas e mental model entre backend e frontend, reduzindo a curva de troca de contexto de quem trabalha nos dois lados.